

FIPS 140-3 Pre-Assessment Submission Package

Module: pqc_crypto_module v0.1.0 **Vendor:** Goya Ledger Project **Contact:** carriagadafalcone@gmail.com **Date:** 2026-08-17 **Target:** FIPS 140-3 Level 1 (software-only)

1. Executive Summary

pqc_crypto_module is a Rust-based post-quantum cryptographic module providing ML-DSA-65 (FIPS 204), ML-KEM-768 (FIPS 203), and SHA3-256 (FIPS 202) for the Goya Ledger distributed ledger platform. The module wraps PQClean reference implementations via FFI and enforces a strict approved-mode state machine.

We are seeking a pre-assessment / gap analysis to identify any issues before formal CMVP submission.

Key differentiators: - All three NIST PQC standards implemented (ML-DSA, ML-KEM, SHA3) - ACVP test vectors produce byte-exact matches against NIST-published expected values - 133 tests across 12 suites including adversarial, malleability, IND-CCA2, and timing checks - Deterministic keygen/signing FFI for ACVP automation already built

2. Document Index

#	Document	File	FIPS 140-3 Section
1	Security Policy	SECURITY_POLICY.md	§4, IG 7.1
2	Module Specification	MODULE_SPECIFICATION.md	§4.1
3	Design Document	DESIGN_DOCUMENT.md	§4.5
4	Finite State Model	FINITE_STATE_MODEL.md	§4.4
5	Key Management	KEY_MANAGEMENT.md	§4.8
6	Self-Test Documentation	SELF_TEST_DOCUMENTATION.md	§4.9
7	Operational Guidance	OPERATIONAL_GUIDANCE.md	§4.10

#	Document	File	FIPS 140-3 Section
8	Configuration Management	CONFIGURATION_MANAGEMENT.md	§11
9	Non-Approved Usage	NON_APPROVED_USAGE.md	§4.3
10	This submission package	LAB_SUBMISSION_PACKAGE.md	—

3. Approved Algorithms

Algorithm	Standard	Implementation	Sizes
ML-DSA-65	FIPS 204	PQClean (pqcrypto-mldsa 0.1.2)	pk: 1952B, sk: 4032B, sig: 3309B
ML-KEM-768	FIPS 203	PQClean (pqcrypto-mlkem 0.1.1)	pk: 1184B, sk: 2400B, ct: 1088B, ss: 32B
SHA3-256	FIPS 202	sha3 crate 0.10	output: 32B
HMAC-SHA3-256	SP 800-185	hmac 0.12 + sha3 0.10	output: 32B

Non-approved (gated, outside boundary): Ed25519, SHA-256, HMAC-SHA256. Blocked in Approved state. Excludable at compile time via `--features approved-only`.

4. ACVP Test Vector Evidence

All tests compare byte-exact output against NIST ACVP-Server published expected results.

Algorithm	ACVP Test	Vectors	Source	Status
ML-DSA-65	keyGen (seed → pk/sk)	3	NIST ACVP-Server ML-DSA-keyGen-FIPS204	✓ byte-exact
ML-DSA-65	sigGen (internal, derand)	3	NIST ACVP-Server ML-DSA-sigGen-FIPS204 tgId=22	✓ byte-exact

Algorithm	ACVP Test	Vectors	Source	Status
ML-DSA-65	sigVer (pk+msg+sig)	10	C2SP/Wycheproof mldsa_65_verify_test.json	✓
ML-KEM-768	keyGen (d,z → ek/dk)	3	NIST ACVP-Server ML-KEM-keyGen-FIPS203	✓ byte-exact
ML-KEM-768	encapDecap (m → ct/ss)	3	NIST ACVP-Server ML-KEM-encapDecap-FIPS203	✓ byte-exact
SHA3-256	KAT	3	FIPS 202 published vectors	✓ byte-exact

ACVP Automation Infrastructure

Deterministic keygen/signing FFI is built and tested: - csrc/keypair_from_seed.c — ML-DSA-65 KeyGen_internal(ξ) per FIPS 204 §5.1 - csrc/acvp_derand.c — ML-DSA-65 Sign_internal with injected rnd - ML-KEM-768 keypair_derand / enc_derand — linked to PQClean's existing deterministic functions

These enable automated ACVP testing harness integration.

5. Test Suite Summary

Suite	Tests	What it validates
acvp_keygen.rs	4	NIST ACVP ML-DSA-65 keyGen vectors
acvp_full.rs	6	NIST ACVP sigGen + ML-KEM keyGen/encapDecap
nist_kat_vectors.rs	5	Wycheproof ML-DSA-65 sigVer (valid/invalid)
pqc_gauntlet.rs	43	Bit-corruption, malleability, IND-CCA2, chi-squared entropy, timing, stress
fips_readiness.rs	8	Approved-mode enforcement, state transitions
approved_vs_legacy.rs	12	Legacy blocking in Approved state
api_boundary.rs	7	No direct crypto imports outside boundary
key_zeroization.rs	4	ZeroizeOnDrop, Debug redaction
no_fallback.rs	4	No fallback to classical algorithms
self_tests.rs	2	Power-on self-test suite
Unit tests (lib)	37	Module-internal correctness

Suite	Tests	What it validates
Total	133	

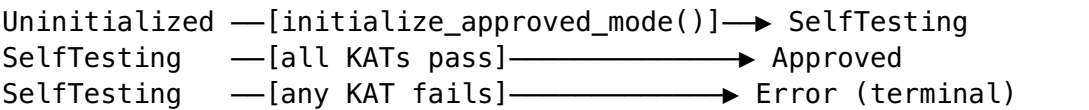
All 133 tests pass. Build environment: rustc 1.97.0-nightly, macOS arm64.

6. Cryptographic Boundary

crates/pqc_crypto_module/		
src/	←	INSIDE boundary (11 .rs files)
api.rs		Single public entry point
mldsa.rs		ML-DSA-65 (FIPS 204)
mlkem.rs		ML-KEM-768 (FIPS 203)
hashing.rs		SHA3-256 (FIPS 202)
rng.rs		CSPRNG wrapper (OsRng)
self_tests.rs		Power-on KATs
approved_mode.rs		FSM (4 states, 3 transitions)
types.rs		Key types with Zeroize
errors.rs		Error types
legacy.rs		Non-approved (gated)
lib.rs		Crate root
csrc/	←	INSIDE boundary (ACVP FFI)
keypair_from_seed.c		Deterministic keygen
acvp_derand.c		Deterministic signing
tests/	←	OUTSIDE boundary
*.md	←	OUTSIDE boundary (documentation)

Enforcement: Rust crate system (private by default) + require_approved() runtime guard + compile_error! for approved-only feature.

7. Finite State Model



3 valid transitions out of 16 possible. Error is terminal — no recovery without process restart. Exhaustively tested (16 transition pairs verified).

8. Entropy Source

Platform	Source	Backend
Linux	getrandom(2)	Kernel CRNG (ChaCha20 since 4.8; CTR_DRBG in FIPS mode)

Platform	Source	Backend
macOS	SecRandomCopyBytes	Fortuna / CTR_DRBG

Continuous RNG test (SP 800-90B §4.3) runs at module startup. Two consecutive 32-byte outputs must differ. Module enters Error state on RNG failure.

9. Known Gaps / Questions for Lab

1. **DRBG wrapper:** Currently delegates to OS CSPRNG. Is this acceptable for Level 1, or do you require a standalone SP 800-90A DRBG within the module boundary?
 2. **Nightly toolchain:** The parent workspace requires nightly Rust. The module itself uses stable features only. Does the nightly compiler affect certification?
 3. **PQClean provenance:** The underlying C implementations come from PQClean (Bernstein/Lange/Schwabe). PQClean is not CMVP-validated. Does the lab require a specific implementation provenance, or is ACVP vector conformance sufficient?
 4. **Ed25519 in Cargo.toml:** Ed25519 is a dependency for legacy backward compatibility but is gated out in Approved mode and fully excluded with `--features approved-only`. Is the approved-only feature gate sufficient, or must Ed25519 be removed from Cargo.toml entirely?
 5. **Algorithm certificates:** ML-DSA-65 and ML-KEM-768 do not yet have CAVP algorithm certificates. Are you accepting PQC algorithm submissions now, or should we wait for CAVP PQC availability?
 6. **Conditional self-tests:** Current self-tests are power-on only. Do you require pair-wise consistency tests for keygen (generate keypair, sign, verify cycle) as conditional self-tests?
-

10. How to Reproduce

Clone and build

```
git clone <repo> && cd goya-ledger
cargo build -p pqc_crypto_module
```

Run all 133 tests

```
cargo test -p pqc_crypto_module -- --test-threads=1
```

Run ACVP vectors specifically

```
cargo test -p pqc_crypto_module --test acvp_keygen
cargo test -p pqc_crypto_module --test acvp_full
cargo test -p pqc_crypto_module --test nist_kat_vectors
```

Run gauntlet

```
cargo test -p pqc_crypto_module --test pqc_gauntlet
```

```
# Clippy (zero warnings)
cargo clippy -p pqc_crypto_module --tests -- -D warnings
```

11. Source Code Access

Full source is available in `crates/pqc_crypto_module/`. We can provide: - Git repository access - Snapshot tarball with SHA-256 checksum - Build reproducibility instructions

We are prepared to enter a pre-assessment engagement at your earliest availability.